

Solution Architecture

Date	17 September 2026
Team ID	SWTID-2026-3976
Project Name	AI FAQ Assistant API
Maximum Marks	4 Marks

Solution Architecture - Layered Component View

Epic-1: Project Architecture

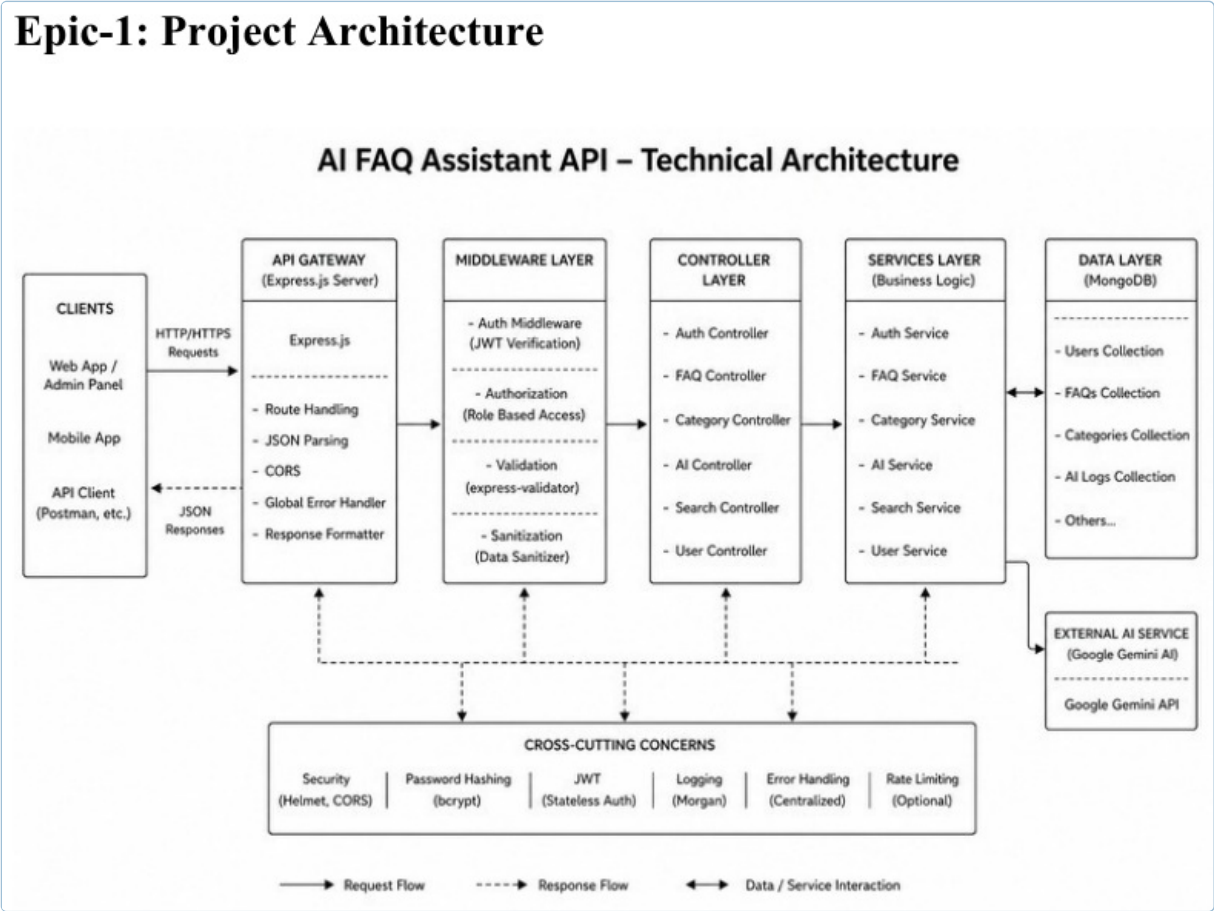


Fig. 1 – AI FAQ Assistant layered architecture, as designed by the team: Clients → API Gateway → Middleware Layer → Controller Layer → Services Layer → Data Layer, with a parallel path from the Services Layer to the external Google Gemini AI service

Component Description

Layer	Responsibility
Clients	Postman / Thunder Client and any future web or admin client issue HTTP/HTTPS requests and consume JSON responses.
API Gateway (Express Server)	src/app.js configures cors, JSON/urlencoded body parsing, a GET / health check, and mounts the /api/auth, /api/faqs and /api/ai route groups; src/server.js is the actual process entry point that connects to the database and starts the listener.
Middleware Layer	src/middleware/authMiddleware.js exports protect (Bearer-token verification, attaches req.user); validationMiddleware.js validates request bodies per route; errorMiddleware.js centralizes error formatting.
Route & Controller Layer	routes/authRoutes.js, faqRoutes.js, aiRoutes.js declare each endpoint's path/method and delegate to controllers/authController.js, faqController.js and aiController.js respectively.
Services Layer	src/services/geminiService.js isolates all Google Gemini calls behind reusable functions, so controllers stay free of AI-vendor-specific code.
Data Layer	models/User.js and models/FAQ.js (Mongoose schemas) plus config/db.js (MongoDB connection) persist and validate all data.

The team's original architecture diagram also sketches Category, Search, AI and User as separate Controller/Service pairs feeding a Data Layer with Users, FAQs, Categories and AI Logs collections. In this cycle's actual implementation the controller layer is three files (authController.js , faqController.js , aiController.js) and the data layer is two Mongoose models (User , FAQ) – category is a fixed enum field on FAQ rather than its own collection, and search is a function inside faqController.js rather than a standalone service. The richer split shown in the diagram is recorded as the Scalability & Future Plan target (see Phase 8).

MVC Design Pattern

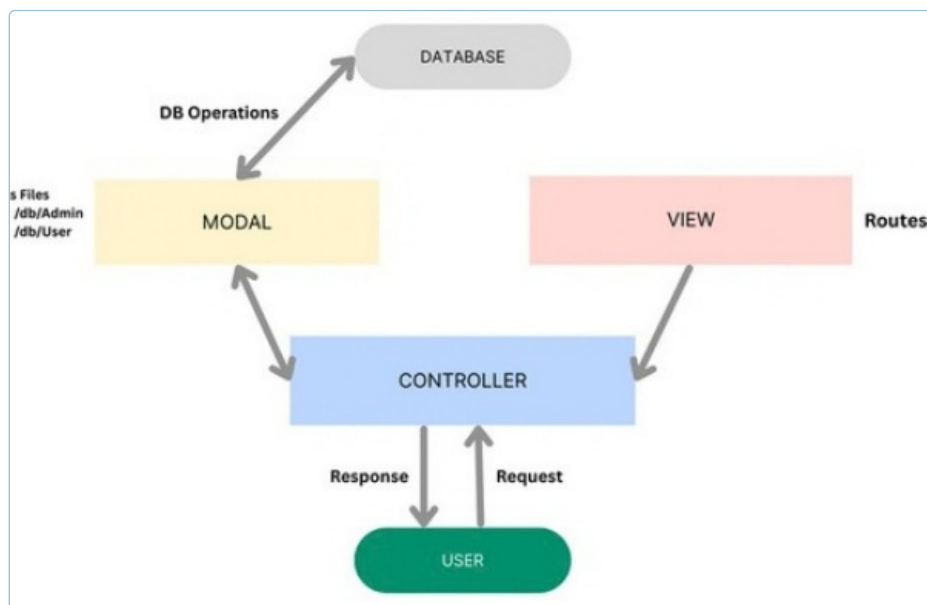


Fig. 2 – MVC mapping used across the codebase (labeled "MODAL" on the team's original diagram; the correct term is **Model**). Database: MongoDB.

The Model layer (`User` , `FAQ` Mongoose schemas) defines structural blueprints and field validation before persistence. The Controller layer is the intermediary: it captures requests from the routing layer, checks constraints, delegates to models or the Gemini service, and packages the result as JSON. There is no server-rendered View layer since this is a pure headless API – the "View" is the API Routing Layer itself (`routes/*.js`), which links each network path directly to its controller handler.